

Milestone C, Architecture Design

System Context:

@startuml

!include

https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.puml

Person(player, "Player", "Interacts with the game via buttons and camera")

Person(admin, "Admin", "Monitors system remotely")

System(system, "Face Frenzy System", "Real-time face counting game")

System_Ext(camera, "USB Camera", "Captures video input")

System_Ext(display, "HDMI Display", "Shows game UI")

System_Ext(buttons, "GPIO Buttons", "User input")

System_Ext(leds, "LED Indicators", "Feedback signals")

Rel(player, system, "Plays game")

Rel(admin, system, "Monitors via HTTP")

Rel(system, camera, "Reads video stream")

Rel(system, display, "Outputs UI")

Rel(system, buttons, "Reads input")

Rel(system, leds, "Controls LEDs")

@enduml

Container:

@startuml

!include

https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml

System_Boundary(system, "Face Frenzy System") {

Container(app, "Game Application", "Python", "Controls game logic and orchestration")

Container(camera, "Camera Subsystem", "OpenCV", "Captures frames")

Container(detection, "Face Detection", "YOLO + FINN", "Detects faces (FPGA accelerated)")

Container(ui, "UI Renderer", "Python + HDMI", "Displays overlays")

Container(io, "I/O Handler", "GPIO", "Handles buttons and LEDs")

Container(db, "Storage", "File/DB", "Stores scores and logs")

Container(monitor, "Monitoring सेवा", "HTTP Server", "Exposes system state")

}

Rel(app, camera, "Gets frames")

Rel(app, detection, "Sends frames for inference")

Rel(app, ui, "Sends render data")

Rel(app, io, "Reads input / controls LEDs")

Rel(app, db, "Stores results")

Rel(monitor, db, "Reads data")

Rel(admin, monitor, "HTTP access")

@enduml

Deployment:

@startuml

!include

https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Deployment.puml

```
Deployment_Node(pynq, "PYNQ-Z2 Board", "Zynq-7000") {

    Deployment_Node(ps, "Processing System (ARM)", "Linux + Python") {
        Container(app, "Game App")
        Container(monitor, "HTTP Server")
    }

    Deployment_Node(pl, "Programmable Logic (FPGA)", "FINN Accelerator") {
        Container(detection, "YOLO Accelerator")
    }

}

Deployment_Node(peripherals, "External Devices") {
    Container(camera, "USB Camera")
    Container(display, "HDMI Display")
    Container(buttons, "GPIO Buttons")
    Container(leds, "LEDs")
}

Rel(camera, app, "Video stream")
Rel(app, detection, "Frame inference")
Rel(detection, app, "Face count")
Rel(app, display, "Video output")
Rel(buttons, app, "Input")
Rel(app, leds, "Output")

@enduml
```

Why ARM (PS) vs FPGA (PL)

- **ARM (Processing System)** handles:
 - Game logic (state machine)
 - UI rendering
 - Input handling
 - HTTP monitoringFlexible, easier in Python
- **FPGA (Programmable Logic)** handles:
 - Face detection (YOLO inference)Massive speedup via parallelism (FINN)

Main loop (Game State Manager)

Detection pipeline (possibly async)

Monitoring server (separate thread/process)